分布式声波测距系统答辩说明

基于 STM32F746G Discovery 的实现原理 代码导读与调试复盘

版本基准：2026 年 11 月 5 日当前工作区。适用对象：准备课程设计答辩的项目成员。

本文根据当前源代码、项目内说明和历次实板调试记录整理。它是帮助理解和讲解项目的技术说明，不是全项目验收通过证明。历史记录中有些描述属于当时的中间版本，涉及协议和算法时以当前代码为准。

我们的系统已经把网络发现、统一时基、声音采集、波形传输、板端相关测距、结果共享、温度补偿和触摸操作串成完整链路。手机标准测试音的固定点结果已有明显改善；拍手在捕获、重复性和方向核对方面仍需完善。电脑负责播放测试音、开发和读取诊断，正常测距计算运行在开发板上。

建议先读第 1 至 4 章建立整体认识，再读第 5 至 9 章理解算法，最后用第 10 至 14 章准备代码提问、演示和答辩。文中的文件路径均相对于项目根目录 `C:\Users\bjtuj\Desktop\final_work`。

1 答辩时先把系统讲清楚

一分钟介绍

可以这样讲：我们使用两块 STM32F746G Discovery 开发板，通过同一个交换机连接。两块板运行同一份固件，上电后自动发现对方并选出主板和从板。系统先利用以太网硬件时间戳建立统一时间基准，再连续采集两块板的麦克风信号。检测到声音后，双方交换带时间标记的短波形，由主板进行相关对齐，得到同一次声音到达两个麦克风的时间差。主板结合温度计算声速，换算距离并判断声源在 A 端还是 B 端，再把同一结果发送到从板显示。我们还加入了拍手与手机音频模式、滤波、自适应门限和多级拒绝规则，降低噪声与错误匹配的影响。

讲完后应补充适用条件：本方法首先得到的是声源到两个麦克风的声程差。只有把声源放在两个麦克风连线的外侧延长线上，声程差才等于两麦克风间距。

当前完成范围

功能	当前实现	需要保留的说明
网络与角色	AutoIP 自动地址、发现、心跳、UID 主从选举	A/B 随选举确定，不是固定的物理左右
时间同步	MAC 硬件时间戳、偏移和漂移估计、失锁检测	还需要示波器证明实际双板脉冲误差
声音采集	48 kHz 双通道 DMA、预触发缓存、起音检测	有波形不等于已形成有效双板测量
测距与共享	主板相关计算、方向、CRC、结果 ACK 和重发	两板结果相同不等于距离真值准确
PHONE 模式	标准脉冲和扫频匹配、周期消歧、多窗口检查	不保证任意音乐和持续纯音可测
CLAP 模式	短促判定、主起音包络匹配	仍有拒绝、额外候选和厘米级误差
温度与操作	DHT11、手动温度、双板共享、触摸切换模式	显示小数位不代表传感器有同等精度
显示与诊断	距离、声源侧、TOA、时差变化量、局部波形	尚未实现双板同步波形叠加显示

2 硬件和软件整体框架

硬件怎样连接

两块开发板用网线接同一个交换机。每块板使用自身音频采集链路和 LCD。手机或音箱只是外部声源，不通过网络告诉板子发声时间。DHT11 模块可以只连接一块板，系统会把有效温度共享给另一块板。ST-Link 用于烧录与调试，不参与正常测距运算。

温度模块为 LY-S0003 上的 DHT11，接线为 + 接 5 V、− 接 GND、S 接 Arduino A0 对应的 PA0。虽然插在 A0，程序用的是数字 GPIO 和定时器输入捕获，不是 ADC 模拟采样。当前配置按 PA0 数字输入条件设计；换板或换引脚时应重新核对电平能力与模块上拉。

示波器验证同步时，使用两块板的 PA15，也就是 Arduino D9，并连接公共地。程序已配置该引脚输出同步脉冲，但仍要通过仪器测量验收，不能用界面上的同步状态替代。

软件按职责分层

最底层是 STM32 HAL、板级 BSP 和外设驱动，负责以太网、SAI、DMA、LCD、触摸及定时器。LwIP 提供 IP、UDP 和 AutoIP。我们在上面编写 app 模块，把网络、时钟、采音、匹配、温度和界面分开。

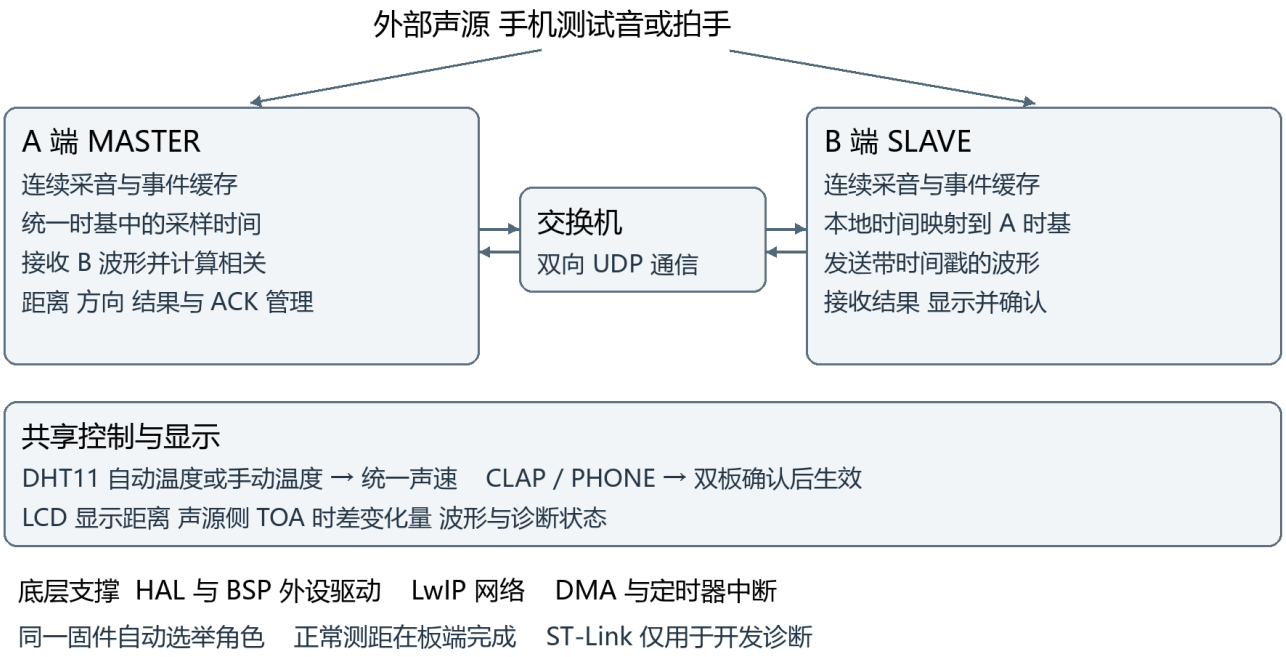


图 1 硬件连接与主要软件职责

主程序采用裸机轮询加中断，没有使用 RTOS 任务调度。DMA 和定时器中断处理需要及时完成的采样和边沿记录，主循环负责网络协议、事件处理、计算和界面刷新。可以把它理解为：中断负责及时记下发生了什么，主循环负责解释数据并协调各模块。

四个 UDP 端口各做什么

UDP 端口	模块	传输内容	作用
50000	app_network 与 app_protocol	发现和心跳	找到对端、确定角色、判断在线
50001	app_clock	SYNC、DELAY_REQ、FOLLOW_UP	建立主从统一时间基准
50002	app_measure	波形片段、测量结果、ACK	交换事件并共享同一计算结果
50003	app_temperature 与设置接口	温度、手动设置、声音模式	同步环境参数和操作状态

当前测量协议版本为 6，温度与模式控制协议版本为 2。部分旧 Markdown 还写着测量版本 3 或 5，那是历史记录；当前接收判断和发送字段在 `app_measure.c` 中均采用版本 6。

3 一次测量从启动到显示怎样完成

上电准备阶段

`main()` 先配置 MPU、HAL、216 MHz 系统时钟和 GPIO，初始化 LCD、网络、时钟、同步脉冲、音频、测距、温度、DHT11 和触摸。初始距离无效，避免把演示数据当成实测值。

`AppNetwork_Process()` 持续处理网络。AutoIP 分配 169.254 网段地址；两板交换发现消息，并比较 STM32 的 96 位 UID。按程序的逐字比较规则，较小 UID 为 MASTER，也称 A；较大 UID 为 SLAVE，也称 B。双方每 200 ms 发心跳，超过 1500 ms 没有收到对端消息就清除在线状态。

网络建立后，时钟模块约每 100 ms 进行一次同步交换。自动温度模式也需要有效温度。网络在线、时钟锁定、设置已确认、温度可用后，系统才允许形成新的有效距离。

捕获和计算阶段

第一步，两板一直采音，并保存声音发生前的历史。声音超过自适应门限后，程序保存触发前后共 3072 点数据。这样即使触发稍晚，缓存中仍有完整起音。

第二步，本地事件先经过峰值与声音形状检查。通过后，带上事件 ID、缓存首点时间、起音时间、模式和 PCM 数据，分六片发送给对端。双方本地事件编号独立递增，不要求两个编号相同。

第三步，MASTER 只在双方缓存完整、时间足够接近、模式一致时尝试配对。它滑动对齐两段声音，并检查相关度、竞争峰、波形形状与允许的时间范围。不能可靠判断时拒绝本次测量，不更新距离。

第四步，主板得到带符号传播时差，结合当前声速算出距离、声源侧和时差变化量。主板立即发送结果，并进行有间隔的重发。从板校验结果和事件 ID，显示后返回 ACK；重复包只确认，不重复记为新结果。

第五步，`main()` 通过 `AppMeasure_PopResult()` 取出结果并刷新界面。失锁、对端离线、温度无效或结果超过约 5 秒时，距离被标成无效。

主循环的实际顺序

```
AppNetwork_Process → AppClock_Process → AppScope_Process
→ AppDht11_Process → AppTemperature_Process
→ AppAudio_Process → AppMeasure_Process
→ 处理触摸 → 取出声音事件 → 取出测量结果 → 按需刷新 LCD
```

注意 `AppAudio_PopTriggerEvent()` 与 `AppMeasure_OnAudioEvent()` 连接采音和测距；`AppMeasure_PopResult()` 连接测距和 UI。这种接口让采样模块不用直接画屏，界面也不用直接计算相关性。

波形和结果包怎样防止混用

一个事件有 3072 个 int16 样本，PCM 共 6144 字节，分成 6 片，每片 512 点。每个波形包共 1056 字节，包含角色、分片号、事件 ID、64 位缓存起始时间、峰值、起音偏移、模式、PCM 和 CRC32。接收端用六位掩码记录完整性，不对缺片补零；每片至少间隔 10 ms，最多轮发三遍，缓存有约 500 ms 配对有效期。

结果包 36 字节，携带主从事件 ID、距离、相关度、削顶标志、采用的声速、J 和参考有效位。ACK 包 32 字节，回传对应的两个事件 ID。主板首发后最多再重发 9 次，收到对应 ACK 停止。CRC32 用于检测报文内容错误，事件和模式检查用于防止把旧包或不相干声音混成新测量；二者解决的问题不同。

4 测距原理和方向含义

为什么时间差能够换算距离

设声源发声时刻为 t0，声源到 A、B 麦克风的距离为 rA、rB，声速为 c。声音到达时刻分别是 tA = t0 + rA/c、tB = t0 + rB/c。两式相减后，未知的发声时刻被消去，所以不需要手机与开发板同步。

定义带符号时差 Δt = tA − tB，则 rA − rB = c × Δt。系统显示 D = c × |Δt|。程序内部主要用纳秒和毫米每秒计算，最后四舍五入为毫米。

例如温度 25 ℃，当前代码声速约为 346.45 m/s。28 cm 的理想传播时差约为 808 μs，73 cm 约为 2107 μs。这里说的是同一次声音到达两板的差，不是两次拍手之间的间隔。

为什么声源位置很重要

把声源放在 A 端外侧、与两个麦克风共线，声音先经过 A 再到 B，这时两条路径之差恰好等于两麦克风间距。放在 B 端外侧同理，只是时间差符号相反。

如果把手机放在两板中间，它到两板距离可能接近相同，系统就会显示接近 0；即使两板相距 73 cm，这个结果也符合声程差原理。若偏离连线，测得的声程差通常小于板距。板边缘间隙也不能当作麦克风间距，卷尺应对准实际麦克风位置。

方向为什么要写 A 和 B

当前代码中 Δt 小于 0 表示 A 较早听到声音，判断声源在 A 端；否则判断在 B 端。这个判断仍依赖外侧连线摆放以及匹配正确。在两板中间或接近等距位置，不能把它理解为完整的空间定位。

A 固定对应当前 MASTER，B 固定对应当前 SLAVE。相同固件通过 UID 选举角色，换掉一块板可能改变原来 A/B 的分配。程序不知道板子在桌面的左边还是右边，因此界面采用逻辑端标识，去掉了容易误导的绝对左右箭头。

5 两块板怎样使用同一个时间基准

不同步会发生什么

两块板即使同时上电，各自计时器的起点和频率也不完全相同。直接把两个本地时刻相减，会把时钟偏移当成声音传播时间。几百微秒的计时误差就会成为明显的距离误差。

我们使用 STM32 以太网 MAC 的硬件时间戳记录同步报文收发时刻，减少软件排队和主循环执行时间对打点的影响。这是基于硬件时间戳的自定义四时间戳同步协议，不应直接称为已实现完整 IEEE 1588 协议栈。

四个时间戳怎样计算

MASTER 发 SYNC 时记录 t1；SLAVE 收到记录 t2。SLAVE 发 DELAY_REQ 时记录 t3；MASTER 收到记录 t4，再通过 FOLLOW_UP 告诉从板相关时间。

在双向链路延迟近似对称的前提下，从板相对主板的偏移估计为 $\theta = [(t2 - t1) - (t4 - t3)] / 2$。往返链路时间 $RTT = (t4 - t1) - (t3 - t2)$。代码拒绝负 RTT 或大于 100 μs 的样本，并对偏移和频率漂移进行估计。

`AppClock_NowNs()` 对外给出统一时间域。从板把自身硬件时间映射到主板时基，其他模块不用分别处理主从差别。锁定门限为 5 μs，超过同步维护期限会失锁；锁定状态只是算法当前判断，不能保证真实引脚误差一定低于该数值。

交换机的排队、链路不对称和时间戳链路本身都可能影响结果。网络在线只说明双方能通信，不代表时钟已同步；同步正常也不代表音频起音或相关匹配一定正确。

怎样用示波器验证

`app_scope.c` 把未来统一整秒时刻映射到本地硬件时间，利用以太网 PTP 目标触发 TIM2，在 PA15 输出脉冲。当前配置在目标触发后约 1 μs 上升，脉宽约 100 μs。两板使用同样配置，比较两路上升沿即可观察实际同步差。

这样可以避免主循环用 GPIO 翻转所带来的不确定执行延迟。应在同一交换机环境下记录至少规定次数的脉冲差并保留截图。诊断中的残差很小、时间戳采用纳秒整数、界面显示小数，都不能替代示波器测量。

6 连续采音和防误触发怎样实现

DMA 和预触发缓存

输入采样率 48 kHz、16 位双通道，由音频 BSP、WM8994、SAI 和 DMA 组成采集链路。DMA 缓冲区有 1024 个 16 位元素，半缓冲为 512 个元素，双通道下是 256 帧，约 5.33 ms。半满和全满回调交替处理数据，CPU 不需要逐点等待采样。

每次事件最终选择较强通道的波形。连续环形历史保留 1024 个单通道样本，约 21.33 ms；整个事件 3072 点，约 64 ms，包含约 42.67 ms 后段。两个板子不必在同一个采样序号触发，因为缓存有各自统一时基的起始时间，相关运算会修正相对位置。

DMA 回调的进入时刻不是最后一个采样点的准确时刻。程序根据 DMA 剩余计数回推处理块边界，再给采样点分配时间。这减少回调延迟直接进入时间标记的影响，但尚不能单独证明所有音频时序误差已经消除。

为什么采用带通滤波

`app_audio_filter.c` 先做 250 Hz 二阶 Butterworth 高通，再做 8 kHz 二阶 Butterworth 低通，使用浮点双二阶 DF2T 结构。高通削弱低频漂移和部分低频环境扰动；低通削弱高频噪声，保留测试脉冲、扫频和拍手中的主要有效成分。

两板以及各通道使用相同系数、独立且连续的滤波状态。这样共同信号经过相同处理，有利于保留相对延迟。滤波不是把所有杂音删除：落在通带内的说话、敲击和反射仍可能存在；两路麦克风及环境的差异也不会因此消失。

不对最终距离简单做平均，是因为错误周期或错误声源匹配不应被混进平均值。当前重点是先判断本次数据是否可信，再决定是否输出；没有把真实板距写进算法，也没有把跳值硬截成目标距离。

自适应门限和等待期

背景幅值估计向上慢、向下较快，并限制单次大瞬态对背景估计的贡献。启动先学习约 1 秒。触发门限取用户原始门限一半、背景倍数加 16、最小幅值 48 三者的最大值；背景倍数 PHONE 为 4、CLAP 为 6。释放门限为触发门限的 75%，形成回差，避免阈值附近反复触发。

强事件之后约 0.8 秒不重新武装，以减少同一次声音的尾音重复触发。弱事件或无有效起音的捕获结束后仅等待约 10 ms，避免轻微噪声占用整个等待期，刚好挡住真正的拍手。用户门限调整也会影响后续结果峰值要求，不是只改屏幕上一条线。

当前 BSP 输入音量设置为 68，较早版本是 80。降增益用于缓解近距离拍手满量程削顶。这个数是 BSP 参数，不能未经标定就当作分贝数。

触发后还要经过哪些检查

第一层是本地采集门限，目的是抓到候选声音；第二层检查结果所需峰值和声音形状；第三层要求双端事件、时间窗口、模式和波形完整性一致；第四层检查相关峰、歧义和传播范围；最后还要保证网络、同步和温度有效。

所以屏幕有波形、EVT 增加、TOA 更新、最终距离更新是不同阶段。前面有变化而后面无结果，可能是合理拒绝，也可能是门限不合适、另一板未捕获或网络传输失败，必须读各阶段计数区分。

7 两种声音模式分别怎样匹配

相关运算的直观解释

把两段波形想成两张透明纸，一张相对另一张左右移动。每移动一格，计算两者有多像；最像的位置就是候选对齐延迟。归一化相关把点积除以双方能量的平方根，减小单纯音量不同对分数的影响，但不能保证严重失真和回声下仍然正确。

普通音频使用 PCM 一阶差分后计算归一化相关。差分强调波形变化，减弱慢变背景影响。候选偏移搜索为 ±960 点，参考窗口为第 1024 至 2047 点。搜索的是缓存相对对齐量，并非直接把这个点数当成最终声程差。

最终 Δt = 本地缓存首点时间 − 对端缓存首点时间 − 对齐延迟。主板的本地就是 A。必须把缓存起始时间与波形相对偏移一起计算，才能处理两端不同触发时刻。

PHONE 怎样处理脉冲和周期错配

手机脉冲中的 2 kHz 载波约每 0.5 ms 重复一次，相关函数可能出现多个相近高峰。错一个周期对应约 17 cm 声程误差，因此不能只选最高峰，更不能认为高相关就必然正确。

普通波形峰明显且唯一时，程序用峰值附近三个点做抛物线插值，估计不足一个采样点的偏移。竞争峰过近时，先用 48 点 RMS 包络及包络变化寻找整体起音，再从附近的载波相关峰中选择，无法明确消歧就拒绝。

PHONE 后续增加两类检查。一是后段能量质量检查，要求测试音后段相对背景和峰值块仍有一定能量，减少短促弱杂音冒充手机测试音。二是在三个相邻的 384 点子窗口中检查同一个候选时差，相关度至少 0.55，局部最优偏移不能偏离候选超过 ±3 点。不同片段要求一致，降低偶然错峰输出。

这也说明 PHONE 适合当前有明确起音且具有一定持续长度的标准脉冲和扫频。连续音乐、无起音纯音、过短提示音不保证通过，手机模式不是通用音频识别器。

CLAP 为什么走包络分支

拍手短、宽带、形状受力度和反射影响大。两端原始波形相位可能差很多，强行按载波细节相关容易错峰。CLAP 先要求触发后前 8 ms 能量超过后段总能量的 50%，且峰值平方超过后段平均能量的 8 倍，作为短促事件候选。

双方均为短促候选时，程序匹配 RMS 包络变化。窗口固定在触发锚点附近第 928 至 1408 点，偏移搜索 ±240 点，相关度门限 0.60，主次峰差至少 0.03。把窗口放在主起音附近，是为了避开预触发噪声和较晚回声。±240 点限制的是局部对齐，不是最终声程差。

当前拍手分支使用整数采样点包络延迟，没有像 PHONE 那样做原始波形亚采样细化。48 kHz 一个采样点约 20.83 μs，在 25 ℃时约对应 7.2 mm；包络形状差异和反射会带来更大的实际误差，因此不能把 7.2 mm 说成实际精度保证。

切换按钮解决什么问题

CLAP 需要短促形状和较强峰值，PHONE 需要更持续的有效后段。使用同一套严苛规则会互相误伤。现在可点击任一板波形区上方的 CLAP 或 PHONE 按钮切换，通过控制协议同步到另一板；默认 CLAP。

切换会清除旧捕获、待配对事件、待发结果和上次时差参考。测量包也携带模式位，拒绝模式不同的旧数据。从板请求丢失时会重试直到主板确认。这个实现是手动选择处理策略，不能表述为已经完成拍手与标准音频的全自动分类验收。

8 温度补偿和共享设置

DHT11 怎样读取

app_dht11.c 使用 TIM5 的 1 MHz 计时。主机把 PA0 拉低约 20 ms 后释放总线，传感器通过不同高电平宽度发送 40 位数据。CH1 和 CH2 捕获上升、下降沿；CH3 结束启动低电平；CH4 提供约 7 ms 接收超时。读取周期约 2.1 秒。

定时器记录边沿，主循环解析温湿度和校验和。它没有为了等待每一个位而长时间忙等或全局关闭中断，因此较少干扰音频 DMA 和网络。湿度可供诊断，目前不参与声速计算。

一块传感器怎样服务两块板

两板运行同一固件，不需要按传感器装在哪里选择程序。每板上报本地有效样本，主板优先选自身传感器，否则选从板传感器。自动模式下超过 3 秒无有效样本，温度不可用并阻止新测量；用户也可以切手动温度继续。

手动温度范围 −20 至 60 ℃，触摸每次调整 0.5 ℃。任一板可以操作，从板请求交给主板统一确认。声音模式设置也复用这套可靠控制机制，接口在 app_settings.h，实现目前放在 app_temperature.c 中，并没有单独的 app_settings.c。

公式和误差怎样说明

当前生产代码为 c = 331.3 + 0.606 × T，单位 m/s；程序保存声速为 mm/s，先统一单位再算距离。温度上升时声速增大，同样传播时差换算出的距离也增大。主板把本次采用的声速放进结果包，避免两板分别取温度导致结果不同。

前期评分要求核对记录使用的是 c = 331.4 + 0.6 × T。两个公式接近，但并不相同，答辩前应按正式任务材料统一实现和文档，不能声称当前代码已完全按指定公式实现。此次编写文档没有修改固件。

温度显示到 0.1 ℃只是数值格式，不代表 DHT11 的绝对精度达到 0.1 ℃。自动温度功能实现、温度驱动读数稳定、温度补偿通过毫米级验收，是三个不同结论。

9 屏幕上每个数到底表示什么

显示项	来源和含义	不能据此直接证明
距离	当前成功配对的 c × 绝对传播时差	任意声源位置下的真实板距
A 或 B	主板按时差符号判断较早到达的一端	桌面的固定左侧或右侧
TOA 与事件号	本板最近一次起音检测的统一时基时间戳	本次已被对端匹配并成功测距
时差变化量 dDT	相邻成功结果的绝对传播时差变化量	同步误差、发声周期、原始传播时差
相关度	两段波形或包络的匹配分数	测距绝对精度一定合格
局部波形	本板采音的简化快照	两板波形已在同一坐标中同步叠加
CLIPPED 等状态	削顶或本次拒绝原因	完整的自动可信度分类系统

时差变化量的最终定义

设第 n 次成功测量的带符号传播时差为 Δt(n)。当前显示的变化量为 J(n) = 绝对值〔绝对值 Δt(n) − 绝对值 Δt(n−1)〕，单位 μs。先对两次时差各自取绝对值，再相减，最后再取绝对值。

例如上次 −800.0 μs、本次 +805.5 μs，显示 5.5 μs，而不是 1605.5 μs。第一次没有上一条参考显示 ---。模式切换、掉线和失锁后也会重新建立参考。拒绝事件和结果重传不更新参考。

主板在纳秒整数上计算 J，再放入结果包共享，界面保留 0.1 μs。这样从板即使漏收过一个中间结果，也不会自行拿不同的上一条计算出另一个 J。

为什么会出现几千微秒

曾经有过显示字段选择不符合最终需求的中间版本；后来按用户要求统一为上述公式。现在公式正确也仍可能出现大数：只要相邻两次被接受的测距结果发生大跳变，J 就会很大。例如历史中 713 mm 和 1090 mm 对应的绝对传播时差变化约 1088 μs。

固定位置出现这种数值应排查错配、噪声、反射、模式和同步，而不是把显示值限制到 100 μs 以下。显示小数位和公式单元测试也不能证明声学重复性已经达标。当前指标按用户最终口径实现，是否满足评分表中到达时刻检测项目，仍应单独核对验收口径。

10 每个文件负责什么

建议的代码阅读顺序

先读 `main.c` 看模块怎么连接；再看 `app_measure.h` 和 `app_audio.h` 理解数据；接着读 `app_audio.c`、`app_measure.c` 理解一次测量；最后读时钟、温度、网络和 UI。第一次不必从 HAL 寄存器和 LwIP 内核开始。

头文件 `.h` 定义公开函数、结构和常量，源文件 `.c` 实现内部状态与处理。其他模块尽量通过公开接口交互，避免到处直接修改内部变量。下表把同名的 `.c/.h` 合并介绍。

自编应用模块

文件	实现功能	阅读时重点看
Core/Src/main.c 与 Core/Inc/main.h	外设与模块初始化，主循环，事件转发，触摸操作，结果有效期和 UI 模型更新	main、App_UpdateNetworkUi、App_UpdateAudioUi、App_UpdateTemperatureUi
app_network.c 与 app_network.h	AutoIP、对端发现、UID 选举、心跳、在线状态与网络诊断	AppNetwork_Init、AppNetwork_Process、角色选择和掉线清理
app_protocol.c 与 app_protocol.h	发现与心跳协议的数据结构、编码解码和合法性校验	不要和测量波形协议混为一谈
app_clock.c 与 app_clock.h	MAC 时间戳、同步交换、偏移漂移估计、统一时间、锁定状态	accept_sample、AppClock_Process、AppClock_NowNs、AppClock_RawDeadlineForGlobal
app_scope.c 与 app_scope.h	以太网硬件目标时间触发 TIM2，PA15 输出同步脉冲	AppScope_Init、AppScope_Process，以及失锁撤销输出
app_audio.c 与 app_audio.h	音频 DMA、双通道处理、预触发环形缓存、起音、等待期和声音事件	audio_process_half、audio_find_onset、AppAudio_PopTriggerEvent、AppAudio_CopyTriggerCapture

文件	实现功能	阅读时重点看
app_audio_filter.c 与 app_audio_filter.h	高低通滤波、背景估计、短促判定、PHONE 后段质量检查	AppAudioFilter_Process、AppAudioFilter_IsImpulse、AppAudioFilter_PhoneQuality
app_measure.c 与 app_measure.h	波形分片、配对、相关和包络算法、距离方向、J、结果 ACK、历史与诊断	AppMeasure_OnAudioEvent、correlate、correlate_envelope、phone_lag_consistent、AppMeasure_Process
app_temperature.c 与 app_temperature.h	温度交换、自动选择、手动控制、超时和声速计算	AppTemperature_Process、AppTemperature_GetSoundSpeed、控制请求确认
app_dht11.c 与 app_dht11.h	PA0 定时器边沿捕获、DHT11 时序、40 位解码和校验	AppDht11_TimerIRQ、AppDht11_Process、AppDht11_Decode
Core/Inc/app_settings.h	CLAP 和 PHONE 模式接口及待确认状态	AppSettings_GetAudioMode、SetAudioMode、IsPending；实现在 app_temperature.c
app_touch.c 与 app_touch.h	读取触摸屏坐标，向上层提供触摸事件	AppTouch_Init、AppTouch_Poll；具体按钮含义由 main 和 UI 解释
app_ui.c 与 app_ui.h	LCD 初始化、缓冲区、界面模型、绘图、温度与门限触摸区域	AppUI_Render；显示层不重新进行相关运算
app_font_zh.c 与 app_font_zh.h	当前界面需要的中文字模及绘制支持	中文字形资源，不是测距算法

除 main 和明确标出路径的文件外，上述源文件位于 `Core/Src`，对应头文件位于 `Core/Inc`。

外设 网络底层和工程配置

文件或目录	职责	与我们实现的关系
Core/Src/gpio.c 与 Core/Inc/gpio.h	基础 GPIO 初始化和引脚定义	部分专用引脚由对应 app 模块继续配置
Core/Src/stm32f7xx_it.c 与对应 .h	SysTick、ETH、DMA2 Stream7、TIM5 等中断入口	把中断交给 HAL 或应用处理函数
Core/Src/stm32f7xx_hal_msp.c	HAL 底层外设时钟和引脚支持	属于外设初始化配套
Core/Src/system_stm32f7xx.c	启动阶段系统和时钟基础支持	具体工作时钟还要看 main 的 SystemClock_Config
Core/Inc/stm32f7xx_hal_conf.h	HAL 功能模块配置	决定启用的 HAL 组件
LWIP/App/lwip.c	LwIP 和网络接口初始化与轮询入口	app_network 在此基础上提供业务逻辑
LWIP/Target/ethernetif.c	MAC、PHY、DMA 描述符、收发缓冲和 netif 驱动	与时间戳和正确传输错误返回密切相关
LWIP/Target/lwipopts.h	LwIP 内存、协议和运行选项	包括固定堆地址等嵌入式约束
Drivers 与 Middlewares	HAL、CMSIS、板级 BSP、LwIP 等基础库	主要是厂商代码，不应把整个库都说成自主编写
Utilities	工程配套显示或资源工具代码	以工程实际引用为准，非核心测距入口
LEDtest.ioc	CubeMX 外设工程配置	不完整代表所有手写 app 逻辑，重新生成前需检查保留区
EWARM/LEDtest.ewp	IAR 工程及参与编译的源文件配置	是否加入工程决定文件是否实际构建
EWARM/stm32f746xx_flash.icf	Flash、RAM、栈和堆的链接布局	当前 CSTACK 为 0x2000，即 8 KB
EWARM/settings 下的 xcl	IAR 调试和下载参数	工作区移动后要核对路径
EWARM/LEDtest/Exe 与 List	固件输出及 map	生成物；map 用于确认变量地址和内存布局

历史 .bak 文件不是当前实现，例如 app_measure_onset.c.bak 不在当前构建路径中；应以 IAR 工程文件和当前 .c 为准。hs_err_pid、replay_pid 等日志不属于测距业务模块。

开发测试脚本和说明文件

tools 下的文件	用途
make_ranging_test_audio.py	生成 48 kHz 单声道 pulses 测试音，约 100 ms 的 2 kHz 脉冲、2 秒周期
make_ranging_chirp.py	生成约 50 ms、1.2 至 6 kHz 的 chirp，2 秒周期
make_history_macro.py	读取当前 map，生成结果历史和传输诊断宏
make_pair_macro.py	生成冻结配对与导出波形的宏，用于离线重放分析
make_temperature_macro.py	生成温度相关 SWD 诊断宏
make_noise_macro.py	生成噪声、候选、拒绝与结果计数诊断宏
make_mode_macro.py	生成声音模式和双板设置状态诊断宏
network_probe.py	网络探测辅助工具；不能替代板端自治测距
test_audio_filter.py	调用实际 C 滤波和形状判定，验证频率响应、相对延迟及合成输入
test_impulse_match.py	提取生产包络匹配函数，检查合成延迟、回声和已保存波形
test_phone_consistency.py	检查 PHONE 多窗口一致性，拒绝错误候选与不一致延迟
test_arrival_difference_change.py	检查 J 的首次占位、绝对值顺序、零变化和复位
test_temperature.py	把实际 C 温度控制模块模拟成两节点，验证共享、重试及模式设置
test_dht11_decode.py	检查 DHT11 数据解码、校验、负温度和非法帧
build_defense_doc.py	根据本文 Markdown 生成 Word 答辩文档和框架图，不参与固件运行

docs/measurement_protocol.md 记录测量协议；temperature_compensation.md 记录温度；audio_noise_rejection.md 记录滤波和多轮拍手调试；audio_modes.md 记录模式及手机改进；

`arrival_time_display.md` 记录 TOA 与 J 的需求变化。`output` 下保存实板日志和阶段统计，应保留版本和测试条件。

Python 测试用于验证局部逻辑，不能替代实板的网络时序、真实声学传播、换设备和静默验收。测试音是开发辅助素材，不应未经确认就声称是课程正式指定音频。

11 实现过程中遇到的问题和解决过程

以下按问题推进顺序复盘，每段都区分处理措施与验证边界。部分早期数据来自不同固件版本，不能合并成当前版本的一次完整验收。

初期拍手距离随音量大幅变化

现象是固定板距时，拍手可能显示 0.4 至 1.0 m，甚至出现数米值。只看阈值跨越时刻容易受音量、波形和回声影响，双板也可能捕获不同的瞬态。不能简单用一个常数偏置修正所有场景。

我们逐步加入预触发缓存、带时间标记的波形交换、归一化相关、包络消歧和质量拒绝，让门限主要负责捕获，最终延迟由双方波形匹配决定。手机音频后来趋于稳定，但这不代表拍手已同时解决。

有波形但测试音不出距离

波形能动只证明采集有输入。曾经存在门限、双方候选类型、匹配歧义和传输等多个可能拦截点。最初依靠屏幕询问很难定位，后来增加 SWD 计数、事件历史、峰值、拒绝原因与配对波形导出。

还生成 pulses 和 chirp 两种已知结构的测试音。chirp 的频率随时间变化，比周期载波更容易区分峰；pulses 能暴露周期错配问题。现在遇到不出数，先定位停止在哪一层，再决定改门限或算法。

方向和实际摆放不一致

一部分问题来自把 A 当作固定左板，而更换硬件后 UID 选举可能变化。处理方式是明确 A=MASTER、B=SLAVE，并用逻辑端提示替代固定左右箭头。

另一些记录确实还存在声源侧未确认或匹配符号异常，不能全部归咎于命名。最新 73 cm 拍手结果全部判断 A，而此前用户曾在 B 端测试，因此仍需要固定到明确的一端，双向对比并保存对应波形。

杂音抢触发 拍手被等待期挡住

弱噪声也可能产生候选，并占用较长抑制期，真正拍手刚好被忽略。我们加入 250 Hz 至 8 kHz 滤波、背景自适应门限、回差，以及弱事件约 10 ms 的快速重新武装。有效结果还有独立峰值要求，避免候选即结果。

曾把结果峰值要求降得过低，静默中出现过误报，因此恢复较强的 CLAP 峰值检查。某一阶段在约 122.7 秒静默中两板均无结果，但后续参数又有变化，不能据此声称当前所有环境都已通过防误触发。

拍手削顶和原始波形相关失败

近距离强拍手出现过满量程峰值，削顶会破坏波形形状。我们把 BSP 输入音量从 80 降到 68，后续一批峰值不再满量程。再为短促声音加入独立包络路径，放宽形状能量占比到 50%，使用不同于手机的相关门限。

继续观察发现，提高或降低门限不能解决多峰歧义。因此把拍手包络窗口固定在主起音附近，避免匹配到预触发扰动或较晚回声。28 cm 的一次测试形成 5 次配对、接受 4 次，最大误差 30 mm；这是改进证据，也是尚未 5/5 成功的证据。

网络异常和不合理诊断计数

一次以太网计数接近无符号上限，链接配置显示 CSTACK 原本只有 1 KB。我们把栈扩大到 8 KB，并检查固定 DMA 区域和栈底标记。后续一段运行标记完整、计数正常，但没有抓到越界发生瞬间，所以栈溢出是当时的怀疑，不能写成已证明的唯一原因。

同时修正底层以太网发送失败的返回处理，方便上层区分成功与失败。ST-Link 附加读取可能暂停 CPU，超过心跳期限时会造成暂时掉线，随后又自动恢复，因此调试器影响也必须与真实交换机或网线故障区分。

主板有结果 从板没有结果

这次有明确的计算与发送链路问题：相关搜索较耗时，主板已有结果缓存，但尚未实际发出；下一轮主循环先发现同步超时，取消待发结果，形成两屏不一致。

我们在相关搜索每 64 个候选偏移之间维护网络和时钟，计算期间禁止新波形分片覆盖正在使用的缓存，相关函数启用高优化，计算结束后检查模式与同步是否仍有效，并立即发送首个结果，再按 40 ms 间隔重发直到 ACK。这样解决的是调度与可靠传输，不是把声学误差平滑掉。

实板曾核对到主板 60 个结果收到 60 个 ACK，共同事件 31 个，两板距离最大差 0 mm。计算仍可能接近 1 秒，应继续关注总响应时间；周期性维护网络并不表示相关计算已经很快。

手机偶发大跳值

28 cm 的一批 PHONE 旧记录为 109 至 365 mm，标准差约 41.03 mm。周期音错误峰与短促扰动可能混入，单个高相关度不足以排除。

为 PHONE 加入持续后段质量与三窗口时差一致性检查后，同一固定点最近 32 个结果为 275 至 278 mm，均值约 276.72 mm，标准差约 0.73 mm，最大绝对误差 5 mm。没有按 280 mm 强制修正，也未加入距离平滑。验证只覆盖当时设备、位置和音量，不代表全量程及换设备都达标。

到达时间显示需求多次澄清

先后讨论过本板 TOA、本板相邻事件间隔、两板绝对传播时差，最终用户要求显示相邻两次绝对传播时差之差的绝对值。我们把公式放在主板成功配对路径中，以纳秒整数计算，并放入结果包共享。

最后一次读取事件 323 的变化量为 14496 ns，屏幕应显示 14.5 μs，与公式一致。这能证明当前字段和单位对应正确；并不能证明声源稳定或测距没有跳变。

调试日志本身也需要验证

C-SPY 的部分诊断消息写到 stderr。早期只用标准输出管道保存的文件可能只有启动标题，没有完整数据；后续合并 stderr 保存并核对历史。文档和统计应引用确有数据的日志。

重新链接后变量地址会变化，所以读取宏必须从当前 map 重新生成。不能拿上次硬编码地址继续写变量，也不能把时间相差很大的两段缓存当成同一次声音配对分析。曾导出过相差约 584 ms 的不同事件，那份数据不能用于声学延迟验证。

12 已有实测证据和不能外推的结论

代表性记录

测试场景	已有记录	可以说明什么
33 cm 手机测试音 历史版本	pulses 一批 32 次为 328 至 332 mm，标准差约 0.91 mm	曾在单一固定点取得稳定结果，不能作为当前版本全量程报告
28 cm PHONE 改进后	32 次 275 至 278 mm，均值 276.72 mm，标准差 0.73 mm，最大误差 5 mm	当时固定点重复性较好，仍有约 −3.3 mm 平均偏差
双板结果共享	一批 31 个共同事件最大距离差 0 mm；另一批共同 16 个也一致	主板结果共享链路工作，不证明两板都测准真值
28 cm CLAP 主起音窗口	5 次配对接受 4 次，264、259、276、250 mm	捕获和匹配已有改善，仍非 5/5 成功
73 cm CLAP 最新一批	631、675、665、706、711、716 mm，最大误差 99 mm	仍有约 10 cm 误差；6 个候选不能等同于 5 次拍手全部正确
73 cm 时差变化量	相邻 J 约 125.716、27.723、118.630、12.742、14.496 μs	当前公式与最后直接读取结果一致，重复性仍有波动
DHT11 与共享	一段连续 106 次有效读取无校验或超时错误；20/30 ℃手动共享后恢复 AUTO	证明驱动和控制链路有实板验证，未证明温补全精度验收

73 cm 最新记录全部判断声源 A/MASTER，实际摆放侧仍需重新核对。不能因为距离落在拍手项目 ±20 cm 内，就把方向、成功率、误触发等条件一起视为通过。

答辩前优先完成的工作

第一优先是冻结一个明确固件版本，记录两板 UID、模式、音量、门限、环境温度、麦克风间距和声源位置。用相同条件重复测试，避免前后搬动设备后把不同真值混在一起。

第二优先是做 A 端与 B 端的固定声源对比，尤其保存拍手失败或方向错误那一次的双板配对数据。需要区分没有捕获、只有一板捕获、类型不一致、相关失败、错峰和结果丢失，而不是继续一味降低门限。

第三优先是按正式评分表完成同一交换机下的示波器同步、规定量程多点精度、重复性、响应时间、静默、杂音、换播放设备和断网恢复。把成功率分母写清楚，同时记录漏检与额外结果。当前没有足够证据声明上述全部通过。

第四优先是统一温度声速公式并完成温度变化对比。如果要做单点标定、0.2 至 5 m 扩展、10 m 测距或同步波形显示，需要新增实现和验证；当前尚未完成。现有最终传播时差限制为 ±7.5 ms，在 25 ℃ 附近约对应 2.6 m 声程差，不能宣称已有 5 m 或 10 m 能力。

最后补齐正式提交的组号、姓名学号、分工、版本、接线图和验收表。本文没有替任何成员编造分工或给出未经完整测试的分数。

13 答辩演示和常见提问

建议的现场演示顺序

先展示两板接同一交换机，启动后自动获得地址、出现 MASTER/SLAVE 和同步锁定。说明角色自动产生，程序相同。再确认温度来源和声速，演示 AUTO 与手动设置，并观察双板同步。

然后量出麦克风间距，把标准测试音放在连线外侧固定位置，切到 PHONE。演示双板距离一致，解释 TOA、相关度和 J。移到另一端时明确让老师看到声源位置，再观察 A/B 判断，移动过程中不要混入精度统计。

接着停止测试音，切 CLAP，先确认两板模式一致，再按固定位置和间隔拍手。讲清目前拍手精度与成功率的边界。最后展示拒绝原因、静默观察或预先保存的合格测试记录；没有完成的项目直接说明待验证。

为什么两板烧录同一个程序还要分主从

相同程序可以读取不同 UID 并自动确定职责。主板统一计算和发布结果，从板参与采音与时间同步并显示共享结果。这样不必维护 A 版和 B 版固件，也避免两板独立计算带来不一致。

为什么不用两个门限时间直接相减

门限跨越位置受音量、形状和噪声影响。预触发波形保留起音，相关运算使用一段共同信号估计相对延迟，比单一幅值跨越更有信息。但它仍可能受周期和多径影响，所以需要质量检查。

为什么不用 TCP

当前协议在 UDP 上保留事件身份、时间戳、校验、超时和结果确认，便于按事件处理和丢弃过期数据。可靠性并非 UDP 自带，而是由分片重复发送、结果 ACK、去重与有效期实现。不能只回答 UDP 快，更应说明丢包怎么处理。

两个屏幕都是同一个数是不是作弊

主板根据双方真实采样计算一次结果，再共享显示，这是分布式采集加集中计算的架构。双板一致验证结果传输与事件对应；是否准确还必须对照卷尺真值。二者是不同测试项目。

为什么手机比拍手稳定

标准音频具有固定时长和重复形状，适合波形相关与多窗口一致性。拍手每次力度、位置和反射不同，主起音形状变化大；当前还主要依赖包络整数点对齐，因此真实误差更大。我们为两者分开策略，但拍手仍需进一步验证。

滤波能完全解决噪声吗

不能。滤波主要抑制频带外成分，带内共同敲击和反射仍可能通过。系统依靠滤波、背景门限、形状、双端相关、歧义检查和状态约束共同决定是否输出。还要统计静默误报、漏检和额外触发，不能只展示成功几次。

为什么显示微秒甚至内部纳秒还会有厘米误差

单位和显示位数不是精度。48 kHz 的采样间隔约 20.83 μs，亚采样估计依赖波形质量；时钟映射、麦克风差异、反射和错峰都会引入误差。纳秒整数用于保存和运算，不表示实际具备纳秒准确度。

怎样证明没有用电脑算距离

波形缓存、相关函数、声速和距离运算都在 `Core/Src/app_measure.c` 及相关板端模块中。Python 只生成测试音、构造局部测试或读取 SWD。演示时断开电脑调试连接、保持板子供电和交换机连接，仍可自治运行；正式验收应保存这种条件下的记录。

我们真正做了哪些工作

基于厂商 HAL、BSP 和 LwIP，我们完成业务层的发现选举、硬件时间戳同步控制、事件缓存与传输、双模式声音匹配、结果确认共享、温度与设置同步、触摸显示及诊断工具。底层库来自厂商，算法和集成修改应按真实文件说明，不能把整个工程目录都算作自写代码。

14 编译 调试与后续维护

编译和关键输出

在项目根目录用 PowerShell 执行下列命令，或在 IAR 中选择 LEDtest 配置构建。首次接手应先确认工具安装位置和工程路径。

```
& 'D:\iar\common\bin\iarbuild.exe' EWARM/LEDtest.ewp -make LEDtest
```

主要输出是 `EWARM/LEDtest/Exe/LEDtest.out`，符号布局见 `EWARM/LEDtest/List/LEDtest.map`。本文是对现有版本的说明，没有重新烧录或修改测距算法。此前各次修复构建记录为 0 错误、0 警告，不等于本文生成时又进行了一轮实板验收。

为什么内存布局也重要

STM32F7 的 DMA 访问能力和缓存一致性需要明确处理。当前音频 DMA 缓冲固定在 0x20047000，并通过 MPU 配置配合；LCD 使用 SDRAM 双缓冲。以太网接收池、LwIP 堆和 DMA 描述符也有固定区域，不能随意增加数组或搬地址。

当前以太网接收池位于 0x20010000，LwIP 堆在 0x20048000，RX/TX 描述符在 0x2004C000 与 0x2004C0A0。链接器栈为 8 KB。这些地址说明当前实现约束，实际检查仍应以本次 map 为准。DMA 不能直接按普通变量方式放在不可访问区域；缓存属性不对也可能让 CPU 与 DMA 看见不同数据。

每次修改后怎样确认没有引入新问题

算法变更先用对应的生产 C 局部测试检查基本逻辑，再编译检查资源与警告，最后在两块实板上验证。协议变更需要两板同时更新。验证时记录模式、网络与同步状态，避免把失锁禁止测距的时段误记为静默防误报成功。

调试读取宏要用当前 map 生成；持续测试最好减少 SWD 暂停干扰。日志需要保存真实数值和测试条件，主从通过事件 ID 配对比较，不要拿不同时间的两个屏幕数值直接相减。

附录 资料定位和答辩前复习要点

本说明的代码依据主要为 `Core/Src`、`Core/Inc`、`LWIP/Target` 和 IAR 链接配置。调试过程依据 `docs` 中的专题记录。手机固定点统计可查 `output/modes/phone_quality_summary.txt`，拍手主起音窗口记录可查 `output/noise/attack_window_claps_a.log` 与对应 B 日志。其他诊断数据以实际含数值的文件为准。

答辩前每位成员至少应能自己解释四件事：声程差为何在规定摆放下等于板距；缓存起始时间为何必须和相关偏移一起算；同步、采集、匹配、共享分别解决哪个问题；当前数据能证明什么、还有什么没有证明。

遇到追问时，用代码入口和一组有条件的实测记录支撑回答。例如手机精度问题可讲 PHONE 后段质量与三窗口一致性，并给出 28 cm 的 32 次结果；拍手问题则诚实说明当前主起音包络方法、4/5 配对接受记录和 73 cm 的最大 99 mm 误差，再说明下一步准备抓失败配对与双向声源数据。